

COMP1324 Algorithmics

Branch and bound

Hikmat Farhat

Branch and Bound

- Branch and bound is a general algorithm design paradigm that can be used to solve optimization problems.
- It is a generalization of the backtracking algorithm.
- Recall that backtracking explores the entire search space to find a solution.
- Branch and bound is a more efficient algorithm that can prune the search space.
- This done by using **upper bounds** and/or **lower bounds** to discard branches of the search tree.

Example: subset sum

- Consider a set of integers A and a target value t .
- A solution to the subset sum decision problem returns true if there is a subset $M \subseteq A$ such that $sum(M) = t$.
- We already have seen a solution using dynamic programming.
- Here we will solve it using backtracking.
- Let $A = \{x_1, \dots, x_n\}$, at each step i , the backtracking algorithm tries two choices: including/excluding x_i from the solution.
- We maintain a current sum S , and if at some point $S = t$ we have found a solution.

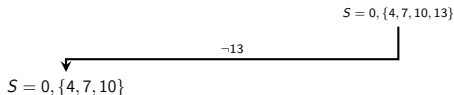
backtracking: subset sum example

- Consider the instance $\{4, 7, 10, 13\}$ with target $t = 22$
- At each step we explore two choices: include/exclude element x_i .

$$S = 0, \{4, 7, 10, 13\}$$

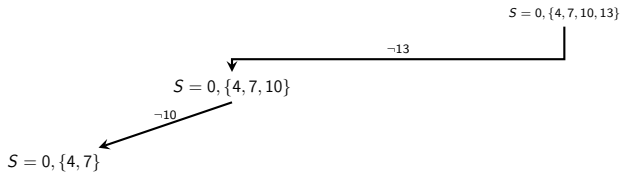
backtracking: subset sum example

- Consider the instance $\{4, 7, 10, 13\}$ with target $t = 22$
- At each step we explore two choices: include/exclude element x_i .



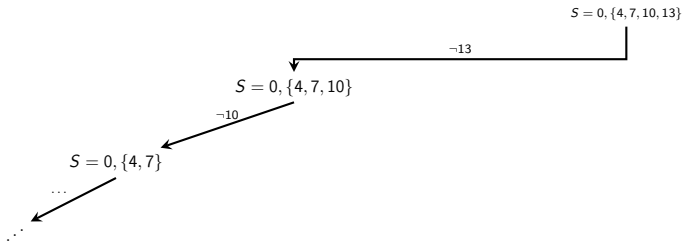
backtracking: subset sum example

- Consider the instance $\{4, 7, 10, 13\}$ with target $t = 22$
- At each step we explore two choices: include/exclude element x_i .



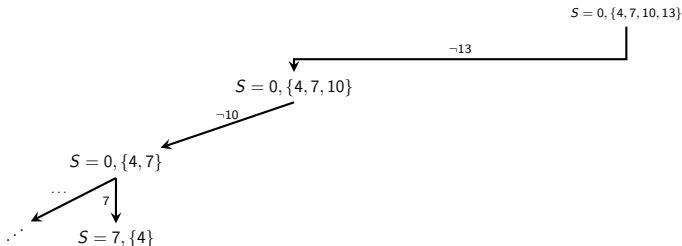
backtracking: subset sum example

- Consider the instance $\{4, 7, 10, 13\}$ with target $t = 22$
- At each step we explore two choices: include/exclude element x_i .



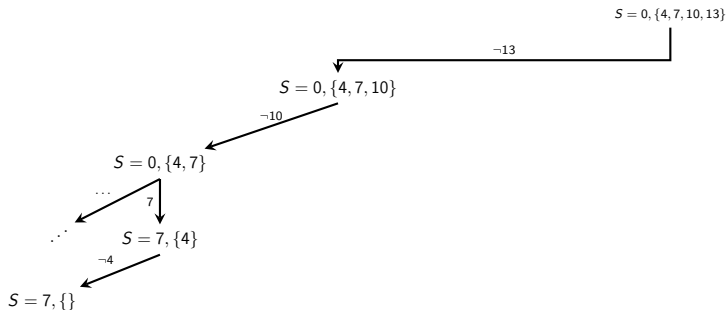
backtracking: subset sum example

- Consider the instance $\{4, 7, 10, 13\}$ with target $t = 22$
- At each step we explore two choices: include/exclude element x_i .



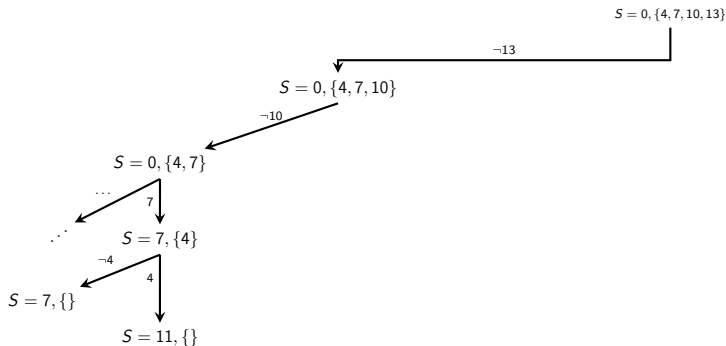
backtracking: subset sum example

- Consider the instance $\{4, 7, 10, 13\}$ with target $t = 22$
- At each step we explore two choices: include/exclude element x_i .



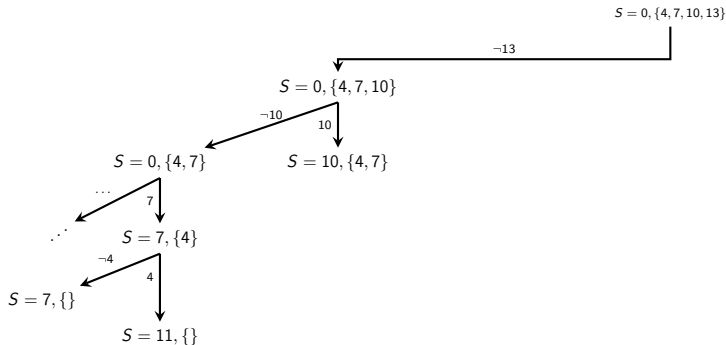
backtracking: subset sum example

- Consider the instance $\{4, 7, 10, 13\}$ with target $t = 22$
- At each step we explore two choices: include/exclude element x_i .



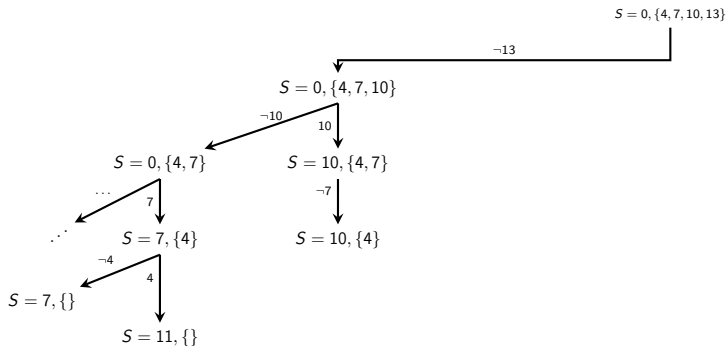
backtracking: subset sum example

- Consider the instance $\{4, 7, 10, 13\}$ with target $t = 22$
- At each step we explore two choices: include/exclude element x_i .



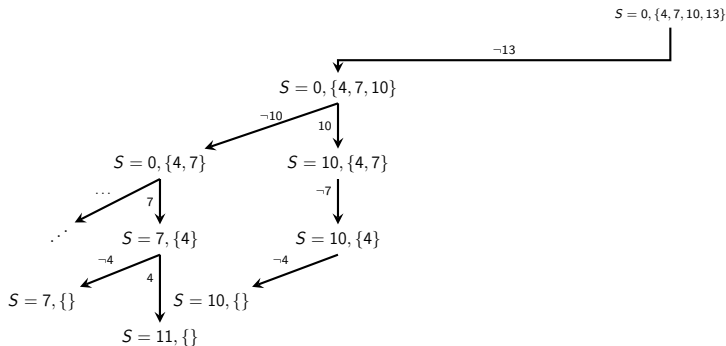
backtracking: subset sum example

- Consider the instance $\{4, 7, 10, 13\}$ with target $t = 22$
- At each step we explore two choices: include/exclude element x_i .



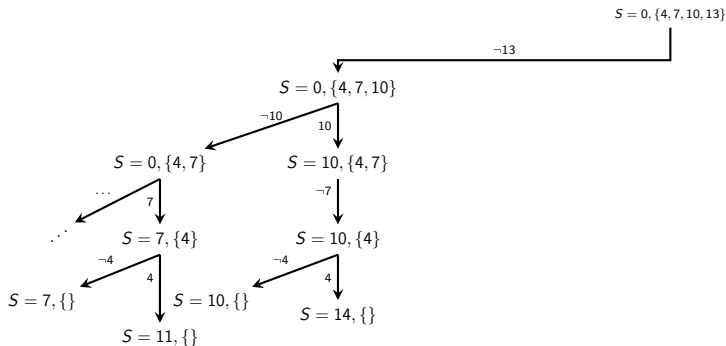
backtracking: subset sum example

- Consider the instance $\{4, 7, 10, 13\}$ with target $t = 22$
- At each step we explore two choices: include/exclude element x_i .



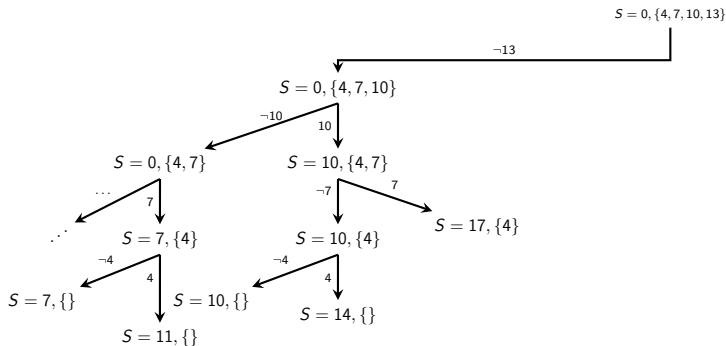
backtracking: subset sum example

- Consider the instance $\{4, 7, 10, 13\}$ with target $t = 22$
- At each step we explore two choices: include/exclude element x_i .



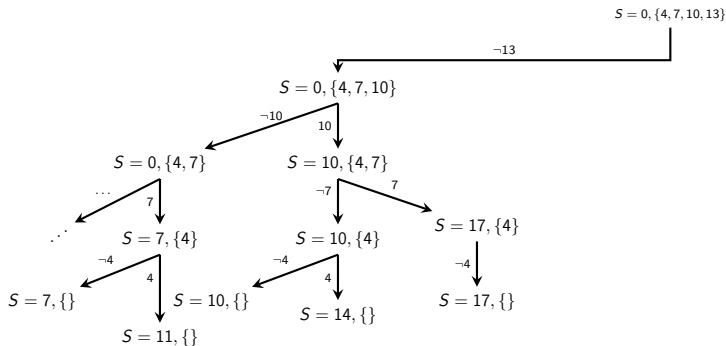
backtracking: subset sum example

- Consider the instance $\{4, 7, 10, 13\}$ with target $t = 22$
- At each step we explore two choices: include/exclude element x_i .



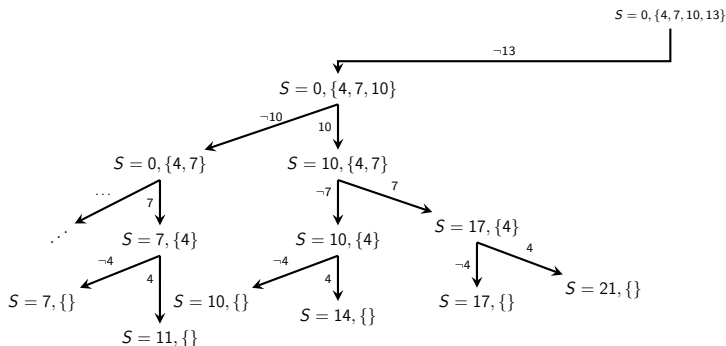
backtracking: subset sum example

- Consider the instance $\{4, 7, 10, 13\}$ with target $t = 22$
- At each step we explore two choices: include/exclude element x_i .



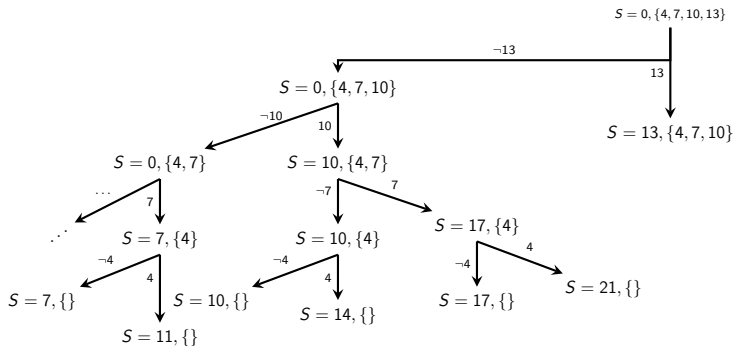
backtracking: subset sum example

- Consider the instance $\{4, 7, 10, 13\}$ with target $t = 22$
- At each step we explore two choices: include/exclude element x_i .



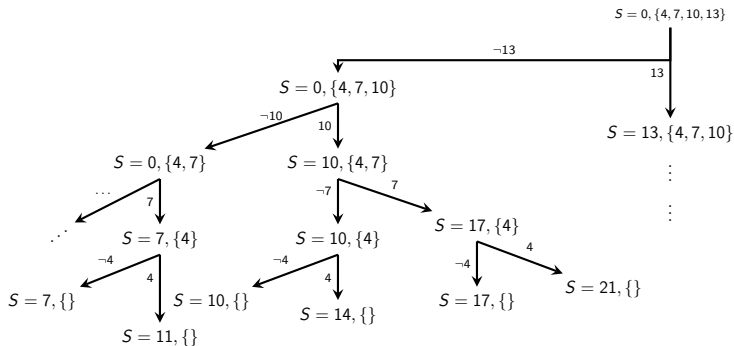
backtracking: subset sum example

- Consider the instance $\{4, 7, 10, 13\}$ with target $t = 22$
- At each step we explore two choices: include/exclude element x_i .



backtracking: subset sum example

- Consider the instance $\{4, 7, 10, 13\}$ with target $t = 22$
- At each step we explore two choices: include/exclude element x_i .



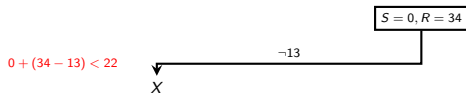
Branch and bound: subset sum

- We can use some bounds to allow us to discard entire branches of the search tree.
- At each step keep two variables: S and R
- As before, S denotes the current sum, and R denotes the sum of all items that **have not been used yet**.
- Initially, $S = 0$ and $R = \sum_{i=1}^n x_i$
- Let t be the target. Given a choice between adding/not adding element x
 - 1 if $S + x = t$ then solution found, return solution.
 - 2 if $S + x > t$ do not include x . backtrack.
 - 3 if $S + R - x < t$ the subbranch without x cannot lead to a solution. backtrack.

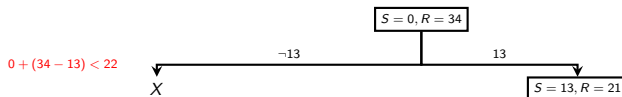
- Consider the instance $\{4, 7, 10, 13\}$ with target $t = 22$

$$S = 0, R = 34$$

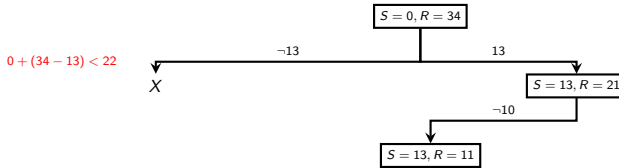
- Consider the instance $\{4, 7, 10, 13\}$ with target $t = 22$



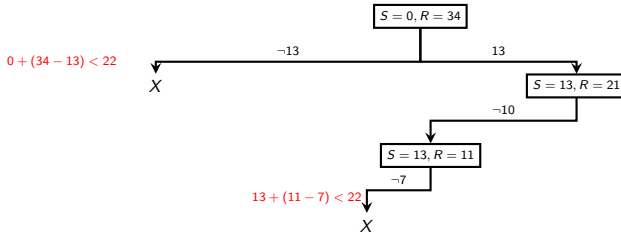
- Consider the instance $\{4, 7, 10, 13\}$ with target $t = 22$



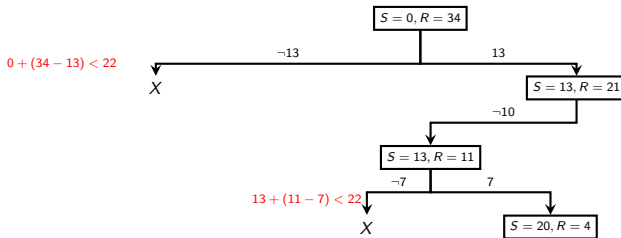
- Consider the instance $\{4, 7, 10, 13\}$ with target $t = 22$



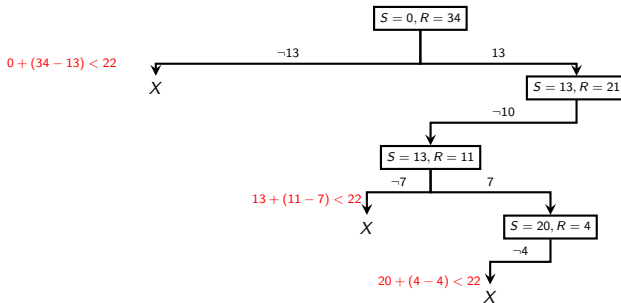
- Consider the instance $\{4, 7, 10, 13\}$ with target $t = 22$



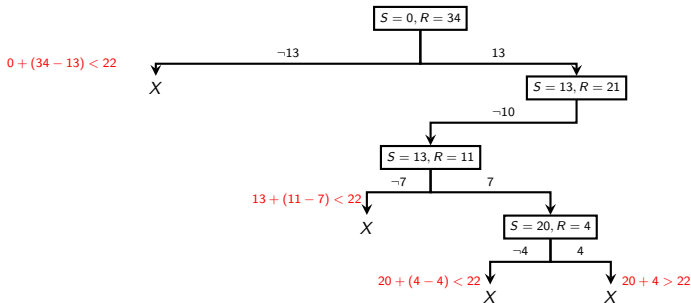
- Consider the instance $\{4, 7, 10, 13\}$ with target $t = 22$



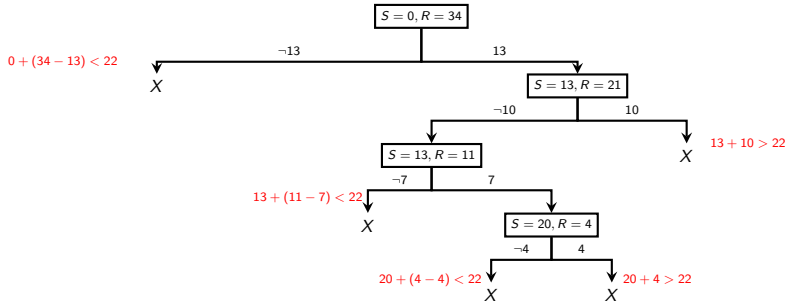
- Consider the instance $\{4, 7, 10, 13\}$ with target $t = 22$



- Consider the instance $\{4, 7, 10, 13\}$ with target $t = 22$



- Consider the instance $\{4, 7, 10, 13\}$ with target $t = 22$



Branch and Bound: Knapsack

- Consider the 0-1 knapsack problem with n items and a knapsack of capacity C
- We compute an upper bound using the greedy solution for the fractional knapsack

$$S = v_1 + v_2 + \dots + v_k + \frac{C - w_1 - \dots - w_k}{w_{k+1}} v_{k+1}$$

- But since

$$\frac{v_1}{w_1} > \frac{v_2}{w_2} > \dots > \frac{v_k}{w_k}$$

- Then we can write

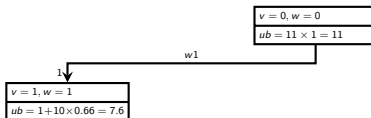
$$S \leq \frac{w_1}{w_1} v_1 + \frac{w_2}{w_1} v_1 + \dots + \frac{w_k}{w_1} v_1 + \frac{C - w_1 - \dots - w_k}{w_1} v_1$$

$$S \leq C \cdot \frac{v_1}{w_1}$$

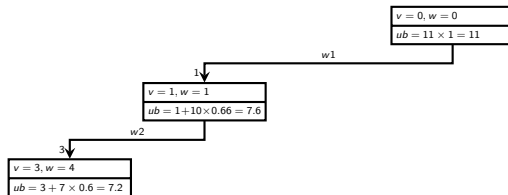
- Assuming that items are sorted by descending $\frac{v}{w}$ we use backtracking with the same order of selection for the items
- At each step, given a knapsack of capacity C and a list of unused items we have an upper bound for the entire branch
- As an example assume we have a knapsack of initial capacity $C = 11$ and the following items

item	Value	Weight	Density
1	1	1	1
2	2	3	0.66
3	3	5	0.6
4	4	7	0.57

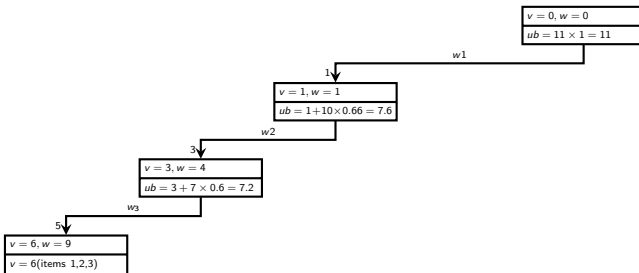
item	Value	Weight	Density
1	1	1	1
2	2	3	0.66
3	3	5	0.6
4	4	7	0.57



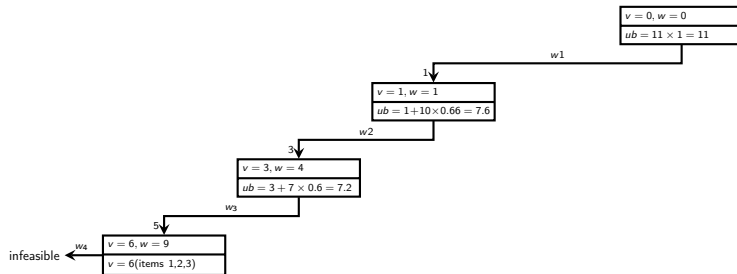
item	Value	Weight	Density
1	1	1	1
2	2	3	0.66
3	3	5	0.6
4	4	7	0.57



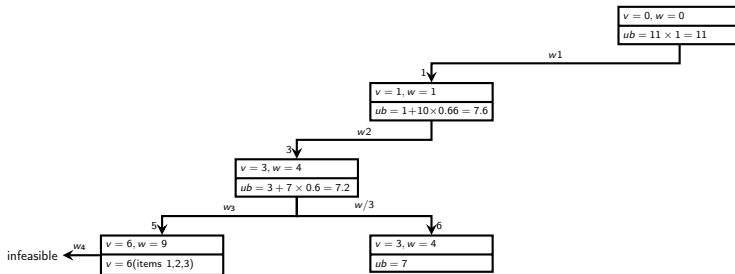
item	Value	Weight	Density
1	1	1	1
2	2	3	0.66
3	3	5	0.6
4	4	7	0.57



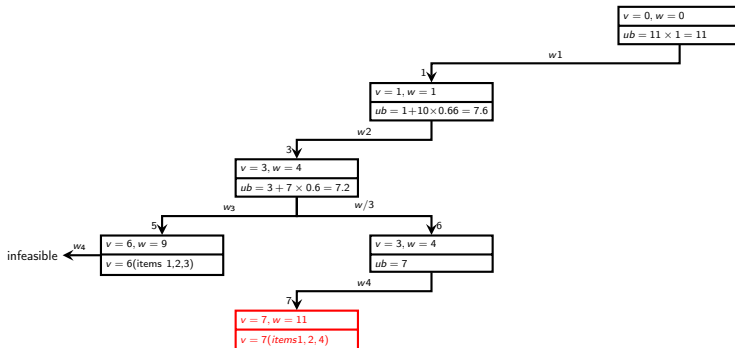
item	Value	Weight	Density
1	1	1	1
2	2	3	0.66
3	3	5	0.6
4	4	7	0.57



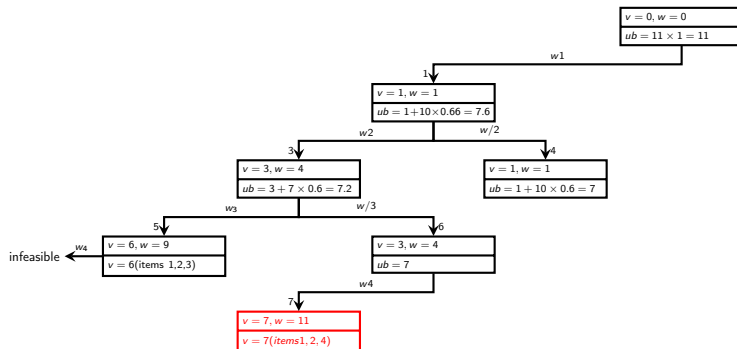
item	Value	Weight	Density
1	1	1	1
2	2	3	0.66
3	3	5	0.6
4	4	7	0.57



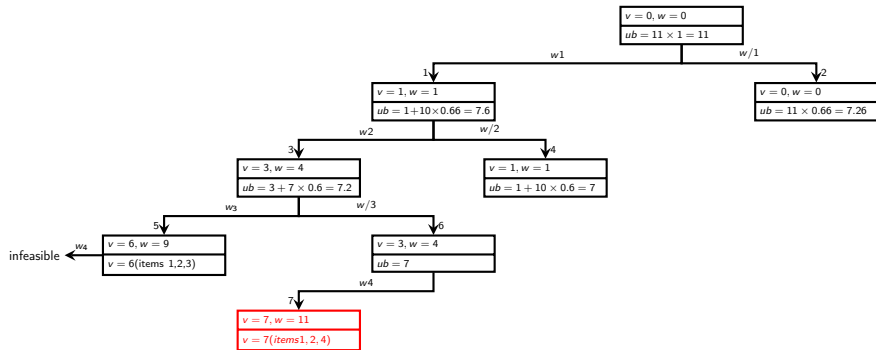
item	Value	Weight	Density
1	1	1	1
2	2	3	0.66
3	3	5	0.6
4	4	7	0.57



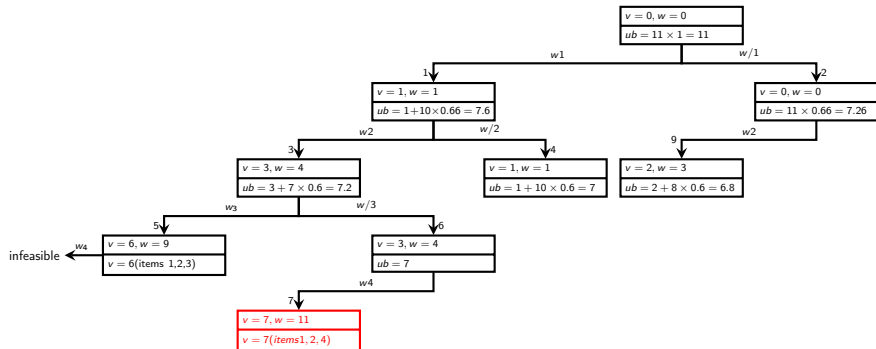
item	Value	Weight	Density
1	1	1	1
2	2	3	0.66
3	3	5	0.6
4	4	7	0.57



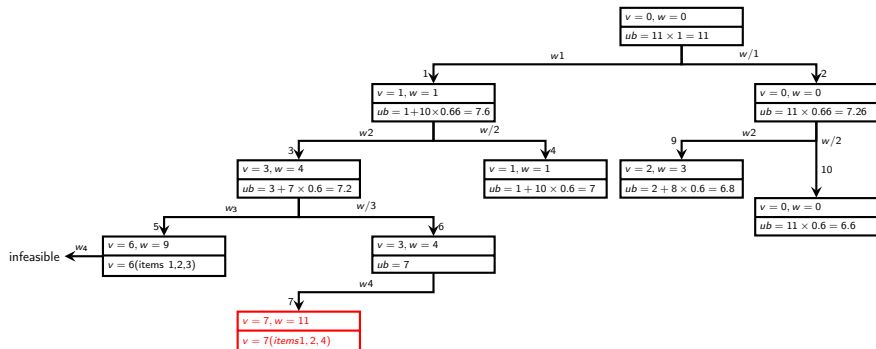
item	Value	Weight	Density
1	1	1	1
2	2	3	0.66
3	3	5	0.6
4	4	7	0.57



item	Value	Weight	Density
1	1	1	1
2	2	3	0.66
3	3	5	0.6
4	4	7	0.57



item	Value	Weight	Density
1	1	1	1
2	2	3	0.66
3	3	5	0.6
4	4	7	0.57



BB for Integer Linear Programming

- Recall that Integer Linear Programming is similar to Linear programming but with the additional constraint that the variables must be integers.
- We develop a general method to solve ILP by using LP relaxation and branch and bound
- For simplicity we consider we use the 0-1 knapsack as an example.
- A 0-1 knapsack instance with n items of values v , weights w , and knapsack capacity B can be formulated as an ILP problem
- **Note:** we use B for the capacity of the knapsack to be in line with LP formulation and avoid confusion with the array c used in the standard form of LP.

ILP formulation

- The ILP formulation is as follows

$$\begin{aligned} & \text{maximise } \sum_{i=1}^n v_i x_i \\ & \text{subject to } \sum_{i=1}^n w_i x_i \leq B \\ & \quad x_i \in \{0, 1\} \end{aligned}$$

- The LP relaxation is given by

$$\begin{aligned} & \text{maximise } \sum_{i=1}^n v_i x_i \\ & \text{subject to } \sum_{i=1}^n w_i x_i \leq B \\ & \quad 0 \leq x_i \leq 1 \end{aligned}$$

Solving the LP relaxation

- The standard form of the LP relaxation

$$\begin{aligned} & \text{maximise } \sum_{i=1}^n v_i x_i \\ & \text{subject to } \sum_{i=1}^n w_i x_i \leq B \\ & \quad x_i \leq 1 \text{ for all } i \\ & \quad x_i \geq 0 \text{ for all } i \end{aligned}$$

- We can incorporate the constraints $x_i \leq 1$ to obtain a form that we are familiar with.
- For example

Example

maximise

$$\begin{bmatrix} v_1 & \dots & v_n \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ \dots \\ x_n \end{bmatrix}$$

subject to

$$\begin{bmatrix} w_1 & w_2 & \dots & w_n \\ 1 & 0 & \dots & 0 \\ \dots & \dots & \dots & \dots \\ 0 & 0 & \dots & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{bmatrix} \leq \begin{bmatrix} B \\ 1 \\ \dots \\ 1 \end{bmatrix}$$

with $x_i \geq 0$ for all i . This is the familiar form of the LP problem.

Taking advantage of LP solvers

- Iterations of the BB algorithm necessitates repeated change in the constraints of the variables
- Luckily, most LP solvers, e.g. linprog in scipy, allow us to specify the bounds of the variables separately.
- For example, consider the 0-1 knapsack instance $v = [9, 6, 4, 9]$, $w = [6, 2, 3, 2]$, and $B = 8$

$$\begin{aligned} &\text{maximise } 9x_1 + 6x_2 + 4x_3 + 9x_4 \\ &\text{subject to } 6x_1 + 2x_2 + 3x_3 + 2x_4 \leq 8 \\ &0 \leq x_i \leq 1 \end{aligned}$$

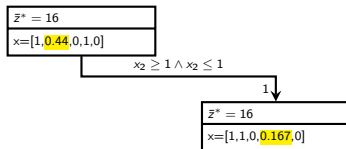
```
from scipy.optimize import linprog
linprog([-9, -6, -4, -9], [[6, 2, 3, 2]], [8], bounds=[(0, 1), (0, 1), (0, 1), (0, 1)])
```

- Let z^* be the minimum value of ILP and $\bar{z}^*$ the corresponding LP relaxation. We have $\bar{z}^* \leq z^*$.
- At every step of the BB algorithm, the *relaxed* LP is solved to obtain a solution z . There are several cases:
 - 1 If $z \geq z^*$ or if the problem is *infeasible* then the instance cannot lead to a better solution. We backtrack.
 - 2 If $z < z^*$ then there are two cases
 - 1 If the value of all variables for the solution are integers we set $z^* = z$ and backtrack.
 - 2 Otherwise, for each variable x_i having value $f \notin \mathbb{N}$ we explore the two branches by including additional constraints: $x_i \leq \lfloor f \rfloor$ in one branch and $x_i \geq \lceil f \rceil$ in the other branch

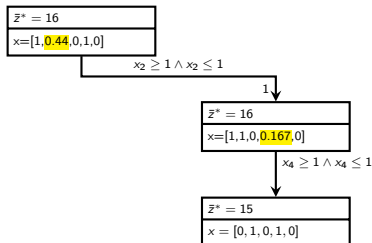
Example: Consider the ILP instance $\max 6x_1 + 9x_2 + x_3 + 6x_4 + 5x_5$
subject to $5x_1 + 9x_2 + 7x_3 + 6x_4 + 9x_5 \leq 15$.

$\bar{z}^* = 16$
$x = [1, 0.44, 0, 1, 0]$

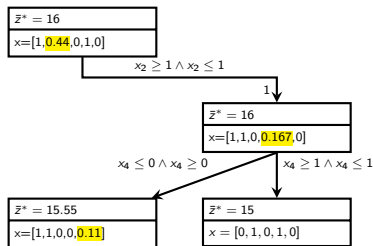
Example: Consider the ILP instance $\max 6x_1 + 9x_2 + x_3 + 6x_4 + 5x_5$
 subject to $5x_1 + 9x_2 + 7x_3 + 6x_4 + 9x_5 \leq 15$.



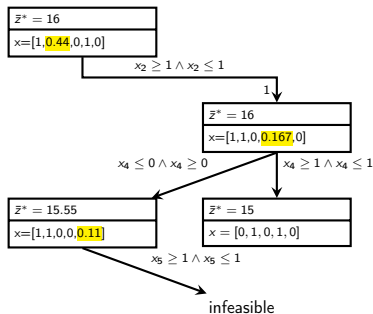
Example: Consider the ILP instance $\max 6x_1 + 9x_2 + x_3 + 6x_4 + 5x_5$
 subject to $5x_1 + 9x_2 + 7x_3 + 6x_4 + 9x_5 \leq 15$.



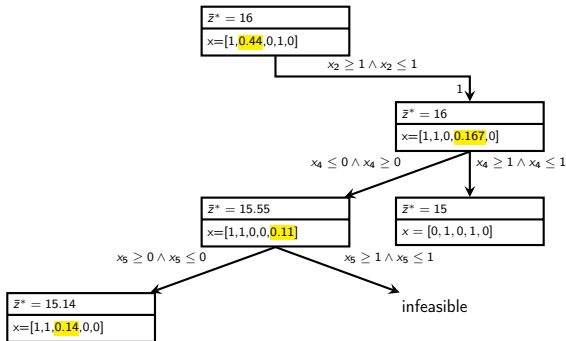
Example: Consider the ILP instance $\max 6x_1 + 9x_2 + x_3 + 6x_4 + 5x_5$
 subject to $5x_1 + 9x_2 + 7x_3 + 6x_4 + 9x_5 \leq 15$.



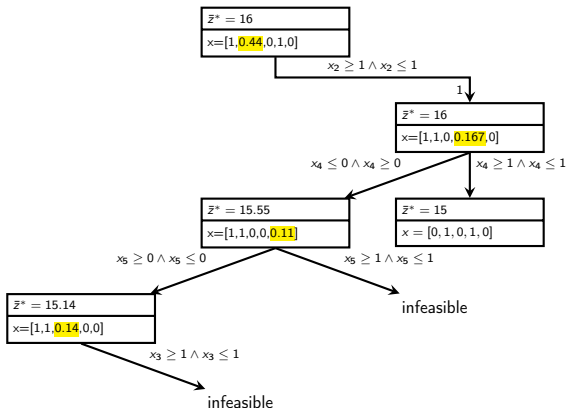
Example: Consider the ILP instance $\max 6x_1 + 9x_2 + x_3 + 6x_4 + 5x_5$
 subject to $5x_1 + 9x_2 + 7x_3 + 6x_4 + 9x_5 \leq 15$.



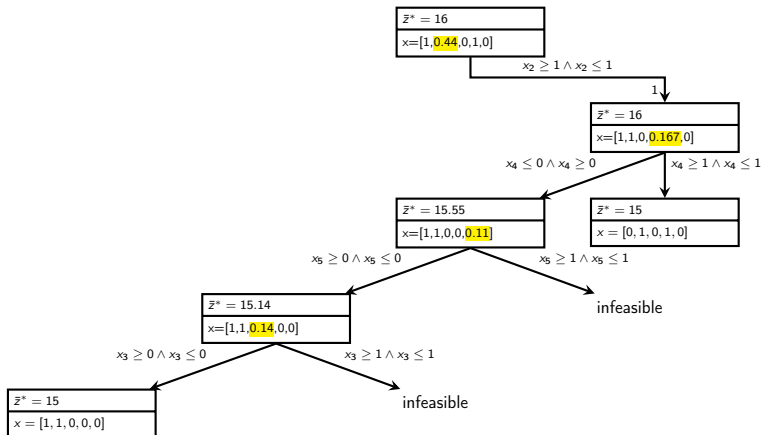
Example: Consider the ILP instance $\max 6x_1 + 9x_2 + x_3 + 6x_4 + 5x_5$
 subject to $5x_1 + 9x_2 + 7x_3 + 6x_4 + 9x_5 \leq 15$.



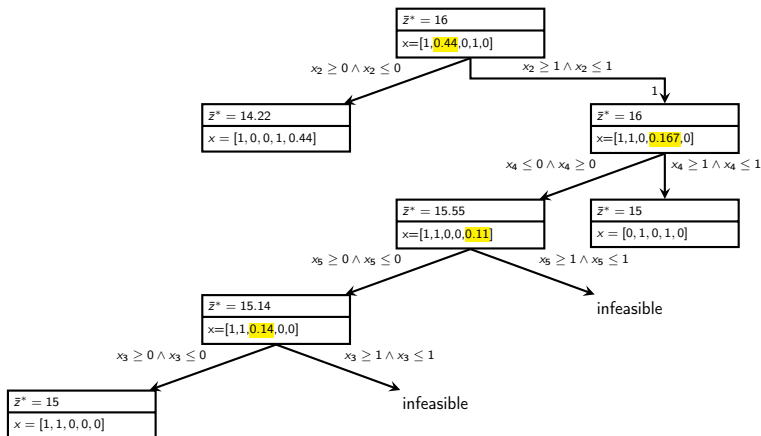
Example: Consider the ILP instance $\max 6x_1 + 9x_2 + x_3 + 6x_4 + 5x_5$
 subject to $5x_1 + 9x_2 + 7x_3 + 6x_4 + 9x_5 \leq 15$.



Example: Consider the ILP instance $\max 6x_1 + 9x_2 + x_3 + 6x_4 + 5x_5$
 subject to $5x_1 + 9x_2 + 7x_3 + 6x_4 + 9x_5 \leq 15$.



Example: Consider the ILP instance $\max 6x_1 + 9x_2 + x_3 + 6x_4 + 5x_5$
 subject to $5x_1 + 9x_2 + 7x_3 + 6x_4 + 9x_5 \leq 15$.



- Branch and bound is covered in section 12.6 of [Introduction to Operational Research](#) by Hillier and Lieberman.
- A Python implementation of branch and bound with LP relaxation for ILP is available [here](#). It is used to solved the knapsack instance covered in the lecture.